20262822《Linux内核原理与分析》第十周作业

真实姓名：姚翔　学号：20262822　整理日期：2026年10月31日
学习内容：教材第 10 章“KVM 及虚拟机技术”、第 11 章“Linux 容器技术”；实验九“Linux 内核学习总结”。

原创声明： 原创作品转载请注明出处。
课程出处： 《Linux 内核分析》MOOC 课程
课程网址： http://mooc.study.163.com/course/USTC-1000029000

一、实验九：Linux 内核学习总结

1. 任务与总结范围

实验九要求用自己的话概括对 Linux 系统和内核的理解，将此前实验报告整理成带引言的目录，并说明学习中最大的收获与遗憾。因此本文沿着已有实验重新组织认识，重点回答“前后几个实验怎样解释同一个程序的执行过程”。

任务页给出的要求也提醒我：总结不能只罗列函数名，还要给读者可继续查阅的入口，并把具体观察转化成自己的解释。

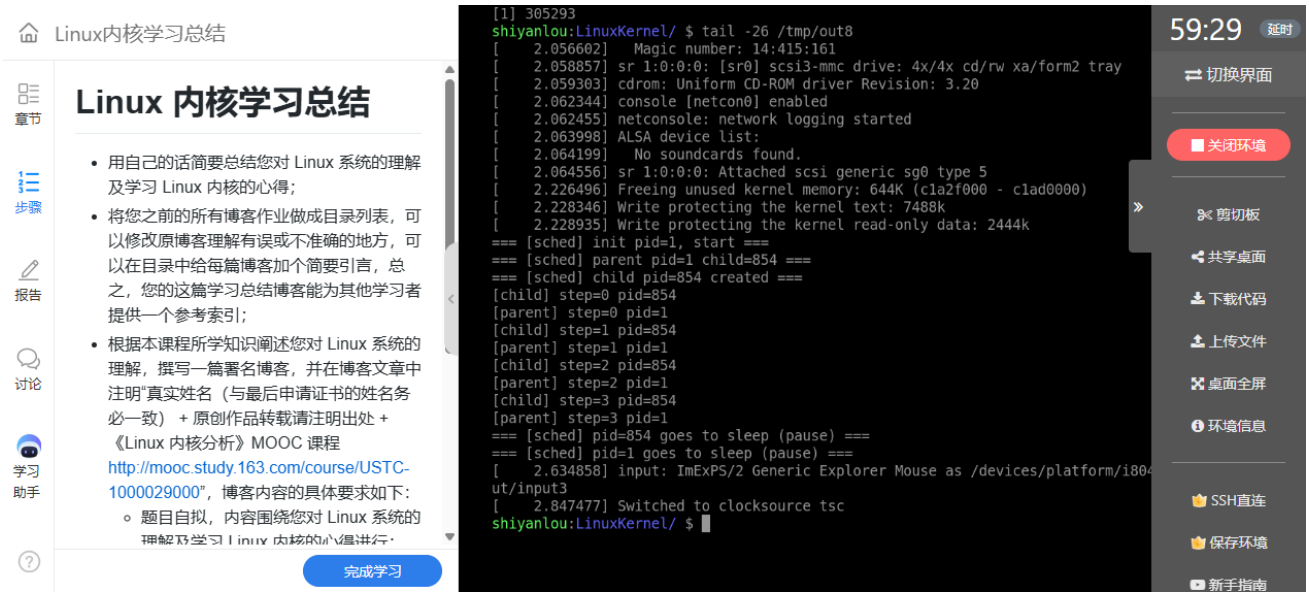


图1： 实验九要求总结内核理解、整理实验目录，并说明收获与遗憾。

2. 我对 Linux 系统的理解

（1）内核管理资源，也维护运行的秩序

从用户的角度，程序执行、文件读写和终端输出似乎是应用自己的行为；从内核的角度，它们涉及 CPU 时间、地址空间、文件对象、权限和设备。Linux 内核通过数据结构记录资源及其关系，通过系统调用提供服务，通过调度让可运行任务获得 CPU，通过中断和异常响应事件。

这里不能把系统调用写成所有用户态转入内核态的唯一途径，异常和硬件中断也会带来控制转移；也不能说内核从不主动运行，启动阶段和内核线程同样执行内核代码。系统调用的特殊之处，是为应用主动请求服务提供约定明确、经过检查的接口。

（2）程序、进程和线程需要分别理解

程序文件描述可以执行的内容，进程具有运行中的地址空间和资源关系，Linux 用任务结构表示可调度实体。线程可以共享地址空间和资源，但各自仍有独立的调度与执行上下文。`task_struct` 是理解这些关系的重要入口，却不是整个进程所有数据都直接放入其中的容器。

实验六和七给出了清楚的对照：fork 产生新任务，exec 在已有进程中装入新程序。实验八进一步说明，任务已经创建并且可运行，也不等于它此刻正在 CPU 上执行；还必须由调度器选中。

（3）内核代码依附于执行上下文

第 9 章指出，所有进程的地址空间在高地址区域共享同一份内核映射。内核不是一个脱离进程的“大进程”，而是中断处理程序和内核线程的集合：前者借用当前任务的上下文执行，后者有自己的 `task_struct` 和内核栈。理解这一点，才能把系统调用、中断、调度和内核线程放在同一张图里。

3. 把实验串成完整的执行过程

（1）启动：从内核初始化到第一个用户程序

实验三跟踪的 `start_kernel`、`rest_init`、`kernel_init` 和执行用户 init 的路径，为后续实验提供了起点。启动并不是普通函数一层层调用到所有进程：初始任务继续承担 PID 0 的 idle 路径，PID 1 与 PID 2 由任务创建机制建立，随后分别执行自身的任务入口。

我需要特别区分“创建了 PID 1”与“PID 1 已经在用户态运行”。前者准备任务，后者还要完成初始化、装载和返回用户态。实验七解释了其中的 ELF 装载与用户现场准备；它与实验三的启动分析形成了衔接。具体函数名要按 Linux 3.18.6 查证，不能把其他版本的 exec 辅助函数名称直接套入。Linux 3.18.6 init/main.c

（2）系统调用：从接口语义到架构协议

实验四在同一进程中用库函数和内嵌汇编取得同一个 PID 884186，说明两种方法可以完成相同的服务请求；它不证明 libc 内部一定执行同一条 `int $0x80`。反汇编中出现 64 位指令和 i386 兼容调用，也让我认识到可执行文件位数与使用的系统调用 ABI 必须分开核对。

实验五把注意力移到内核端：入口保存现场，按调用号选择服务，将返回值写回保存区域，检查返回前工作，最后恢复用户执行。首次 write 断点来自内核解包根文件系统，这件事说明“看见服务函数”与“证明一次用户态入口”之间还需要上下文证据。

我现在会把一个系统调用拆成四个问题：用户提出什么请求、按什么 ABI 传递、内核如何执行、如何把结果交回。这样既能理解接口，也能容纳不同架构和版本的实现差异。

（3）fork 与 exec：创建运行者和替换运行内容

比较项	fork，实验六	execve，实验七
主要作用	创建新的任务与资源关系	将当前进程执行内容替换为新程序
本次身份变化	PID 1 创建 PID 864	执行前后 PID 均为 1
关键现场准备	子任务返回值清零，准备首次内核入口	设置新用户入口与用户栈
关键观察	thread.ip 对应 ret_from_fork	新 IP 对应 hello 的 ELF 入口
用户代码如何继续	父子分别从 fork 之后继续	成功后不回到旧程序的后续语句

实验六中，一个容易误读的现象是 `copy_thread` 里的新描述符还显示父任务复制来的 PID。沿着初始化顺序检查后，才能知道当时尚未赋予最终身份；真正运行到用户态的子进程为 PID 864。这比单看某个字段的截图更能说明源码顺序的重要性。

实验七则把文件与寄存器联系起来： `readelf` 给出的静态程序入口是 `0x08048d0a`， `start_thread` 最终把保存的用户 IP 改成同一个值，新栈为 `0xbfef0580`。返回用户态时使用了新现场，所以 CPU 执行新程序的启动代码，而不是旧程序中 execve 后的正常语句。

这些结论也有边界：普通 fork 的独立地址空间可以采用写时复制，并不要求立即复制所有物理页；exec 通常保留未设置 close-on-exec 的文件描述符；动态 ELF 还可能先进入解释器，不能把静态目标的路径推广成所有程序的唯一过程。

（4）调度：从“轮流运行”到具体的栈交接

实验八的控制台中父子进程交替输出，GDB 则把“轮流”解释为实际的任务选择和上下文切换。`schedule` 组织调度， `__schedule` 选择任务， `context_switch` 处理切换， `switch_to` 保存旧任务恢复点并使用新任务的栈。

最直观的数据是三个 `__switch_to` 入口现场：ESP 都比 next 保存的 SP 小 4 字节。这对应宏手工压入的一个 32 位返回地址，也解释了为什么使用 `jmp` 进入 C 辅助函数后，仍然能够通过返回继续 next 的执行。这个关系依赖入口断点位置，继续执行函数序言后就不能原样套用。

我因此不再把“系统调用进入内核”“中断打断当前执行”和“切换到另一个任务”视为同一件事。它们可能前后衔接，但各自保存什么状态、是否更换任务以及什么时候恢复，都需要单独分析。

（5）一般执行过程：把前面四步合成一张图

第 9 章把上述过程整理为：任务运行 → 事件进入内核 → 保存现场 → 处理 → 返回前调度 → 切换或恢复 → `iret` 返回。5 种特殊情况则说明 fork 的首次执行、exec 的现场替换和内核线程的切换分别改变了哪一步。这张图让我能够把前面每个实验的断点放回同一条时间线，而不是把章节当成互不相关的知识点。

4. 以前的实验与作业目录

以下链接指向仓库内的正式报告，每一项的引言说明可以从中查看什么。综合作业目录下同时提供同名 PDF；独立实验报告保留各自的 PDF 与原始截图。

阶段	作业入口	独立实验报告	阅读引言
先修实践	Linux 命令实践	—	用终端操作理解文件、目录、权限和命令工具，为后续实验准备环境知识。
实验一	第二周作业	从 C 函数到 32 位汇编	结合函数调用分析参数、返回地址和栈帧，是阅读切换代码的基础。
实验二	第三周预习作业	实验二报告	通过教学内核分析时钟中断、任务结构和轮转切换。
编程补充	Linux 命令行参数实践	—	理解 argc、argv 及程序启动参数，衔接 exec 实验。
编译补充	GCC 编译与库实践	—	认识编译、链接和库文件，为 ELF 和动态链接提供背景。
实验三	第四周作业	实验三报告	追踪初始化、PID 0/1/2 和用户 init 启动的关系。
实验四	第五周作业	实验四报告	从同一进程的 PID 比较出发，检查调用号、寄存器与兼容 ABI。
实验五	第六周作业	实验五报告	分析保存现场、分发、返回值与返回前调度，区分内部调用与用户请求。
实验六	第七周作业	实验六报告	解释任务构造、资源关系、子进程返回值和 ret_from_fork。
实验七	第八周作业	实验七报告	用 ELF 文件、动态链接和 start_thread 解释新程序如何开始执行。
实验八	第九周作业	实验八报告	分析调度时机、换栈汇编，以及三次切换入口的栈指针关系。
实验九	本文	实验九报告	将前面各实验整理成可交叉查阅的学习路径。
扩展实践	MenuOS 扩展作业	—	查阅命令扩展的目标、实现与相关文档。

各综合作业目录下同时提供同名 PDF 与正文引用的截图；独立实验报告保留各自的 PDF 与原始截图。

5. 最大的收获：形成可以核验的解释方法

对我而言，最大的收获是能把“程序出现了什么现象”继续追问为“哪个任务、哪份现场、哪条路径导致它发生”。例如，同样是返回用户态，普通系统调用交回服务结果，fork 的子进程交回 0，exec 却使用新的程序入口。共同点是恢复执行现场，差别在于谁准备了这份现场、哪些字段被修改。

这套方法可以落实为四步：先限定内核版本和程序位数；再找会触发的具体操作；随后通过源码、符号、寄存器和调用栈相互核对；最后给结论注明成立条件。优化导致的 `optimized out`、特殊栈帧和不完整回溯，都提醒我不能只依靠一个调试器输出下判断。

我也更重视反例。两个独立程序的 PID 不同，并不否定两种接口语义一致；首次命中 write，不一定来自用户态；yield 后有机会切换，不等于保证轮流；构造中的任务字段，也不能当作最终身份。能解释这些差异，比背下一个固定调用链更有价值。

6. 最大的遗憾与后续计划

这轮学习主要围绕 Linux 3.18.6 的单核 i386 环境展开。它适合观察寄存器和栈，但我还没有把这些结论系统地放到多核并发、任务迁移以及较新 x86-64 内核中比较。许多在单核截图中看起来顺序明确的动作，到了多

核需要进一步考虑锁、内存顺序和每 CPU 数据。

另一个遗憾是，目前对真实内核主体的学习以观察和跟踪为主，还缺少一组由小幅修改、重新构建和对照结果组成的验证。例如可以先增加可控的跟踪点，观察一次阻塞和唤醒，再对照已有的 yield 路径；后续再尝试最小系统调用或其他安全的教学修改，而不是直接破坏关键栈字段。

我希望按下面的顺序继续：先在相同镜像中固定复现已有结果，再比较两核运行下的任务选择和迁移，最后迁移到较新的 64 位内核。每一步都保留源版本、配置、触发程序与输出，让“理解了”能够由下一次实验重新验证。

二、教材第 10 章学习：KVM 及虚拟机技术

1. 阅读范围与问题主线

本周阅读第 10 章“KVM 及虚拟机技术”，印刷页 **291—308**，对应扫描 PDF 的第 **310—327** 页。教材把虚拟化拆成 CPU、内存、I/O 三类资源，再以 KVM、QEMU-KVM 和 StratoVirt 为例说明实现方式。

教材部分	印刷页	本周重点解决的问题
10.1 虚拟机技术概述	291—296	CPU、内存、I/O 分别怎样虚拟化？ESXi、Xen、KVM 有何不同？
10.2 KVM API 的使用方法	294—300	一个用户态程序怎样创建 VM、vCPU 并进入客户机？
10.3 QEMU-KVM 的实现原理	300—302	KVM 与 QEMU 各承担什么？VM exit 后谁处理？
10.4 StratoVirt	302—308	轻量级虚拟化在架构上做了哪些取舍？

2. 问题一：CPU、内存、I/O 分别怎样虚拟化？

疑问与原因。 “虚拟机”看起来像在物理机上再跑一台完整的计算机；如果每条指令都要软件模拟，性能无法接受。三类资源的虚拟化方式并不相同。

理解与解决。 可以按“是否依赖硬件辅助”来理解：

资源	核心问题	典型做法
CPU	客户机指令怎样在物理 CPU 上执行	硬件辅助（VT-x/SVM）：客户机在非根模式运行，特权指令触发 VM exit
内存	客户机物理地址怎样映射到宿主机物理地址	影子页表，或 EPT/NPT 二级地址翻译
I/O	客户机怎样访问设备	QEMU 设备模拟；virtio 半虚拟化；设备直通

CPU 虚拟化的关键是“大多数指令直接执行，敏感指令陷入”。客户机不是每条指令都被解释，而是在 CPU 的非根模式（non-root）下运行；遇到需要 VMM 介入的操作，CPU 触发 VM exit，VMM 处理后通过 VM entry 返回客户机。这样既保持接近原生的速度，又保证隔离。

3. 问题二：KVM 是内核模块，为什么还需要 QEMU?

疑问与原因。 既然 KVM 能创建 VM 和 vCPU，为什么还要一个用户态的 QEMU？两者是否重复？

理解与解决。 分工不同：

- **KVM**：内核模块，利用 CPU 虚拟化扩展提供 `/dev/kvm` 接口，负责创建 VM、创建 vCPU、设置内存槽、执行 `KVM_RUN` 进入客户机，并在 VM exit 时返回用户态；
- **QEMU**：用户态程序，负责设备模型、磁盘/网络/显示、固件加载、迁移等；每个 vCPU 通常对应一个线程，线程里循环执行 `KVM_RUN`。

```
QEMU 进程
  ├─ vCPU 线程 0 → ioctl(KVM_RUN) → 客户机执行
  │                    ↑ VM exit（I/O、异常等）
  │                    └─ QEMU 处理设备/事件
  ├─ vCPU 线程 1 → …
  └─ 设备模型、块/网络、迁移
```

所以 KVM 提供“CPU 虚拟化的内核接口”，QEMU 提供“一台机器所需的设备和服务”。没有 QEMU（或其他 VMM），KVM 只是能运行客户机指令的接口，无法构成完整的虚拟机。

教材 10.2 用一个最小示例说明这套接口的调用顺序，可概括为：

```
kvmfd = open("/dev/kvm", O_RDWR);
vmfd  = ioctl(kvmfd, KVM_CREATE_VM, 0);
vcpufd = ioctl(vmfd, KVM_CREATE_VCPU, 0);
ioctl(vmfd, KVM_SET_USER_MEMORY_REGION, &mem);   /* 客户机物理内存槽 */
run = mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_SHARED, vcpufd, 0);
ioctl(vcpufd, KVM_RUN, 0);                        /* 进入客户机，VM exit 后返回 */
```

`KVM_RUN` 返回后，用户态程序读取 `kvm_run` 中的退出原因，处理 I/O 或异常，再次调用 `KVM_RUN`。这与“系统调用返回后检查返回值再继续”的编程模型有相似之处，但两者所处的执行层级不同。

4. 问题三：VM exit/entry 与系统调用有什么相似之处?

疑问与原因。 两者都涉及“从一种执行环境切换到另一种，处理后再返回”，容易类比，但也容易过度类比。

理解与解决。 相似点在“受控转换 + 现场保存 + 返回”：

维度	系统调用	VM exit/entry
发起者	用户程序	客户机中的敏感指令/事件
转换	用户态 → 内核态	客户机（非根）→ VMM（根）
处理者	内核服务函数	KVM/QEMU
返回	`iret` 回用户态	VM entry 回客户机
现场	`pt_regs`	VMCS 等虚拟化控制结构

但两者不是同一层机制：系统调用是操作系统提供给应用的接口，VM exit 是 CPU 虚拟化扩展提供的硬件行为；前者切换特权级，后者切换“根/非根”操作模式。把 VM exit 说成“客户机的系统调用”会模糊这层区别。

5. 问题四：内存虚拟化为什么比 CPU 虚拟化复杂?

疑问与原因。 CPU 可以直接执行客户机指令，内存却多了一层地址：客户机虚拟地址 → 客户机物理地址 → 宿主机物理地址。

理解与解决。 因为客户机操作系统自己也在管理页表，VMM 不能简单地把客户机页表当成宿主机页表使用：

- **影子页表**：VMM 为客户机维护一份“客户机虚拟地址 → 宿主机物理地址”的页表，客户机修改页表时 VMM 同步更新，开销较大；
- **EPT/NPT**：硬件提供第二级地址翻译，客户机页表负责第一级，VMM 配置第二级，CPU 自动完成两级翻译，减少 VM exit。

KVM 通过内存槽（memory slot）把客户机物理地址区间映射到宿主机的用户态内存（HVA）。内存虚拟化的难点不只是“翻译”，还包括脏页跟踪、内存回收、迁移和 overcommit，这些都与第 5 章的地址空间、页表、TLB 知识相连。

6. 问题五：StratoVirt 与 QEMU 的差别在哪里?

疑问与原因。 两者都能运行 KVM 客户机，教材为什么还要介绍 StratoVirt?

理解与解决。 教材把 StratoVirt 定位为轻量级虚拟机实现，强调启动速度、内存开销和组件化架构；QEMU 则是功能完整的通用 VMM，设备模型丰富、生态成熟。取舍可以概括为：

维度	QEMU	StratoVirt
目标	通用、设备丰富、兼容性强	轻量、快速启动、资源开销低
设备模型	完整模拟	精简，按场景选择
适用	通用虚拟化、开发测试	云原生、函数计算等轻量场景

这不是“谁替代谁”的问题，而是不同场景的工程取舍。理解这一点，比记住某个实现的模块名更重要。

三、教材第 11 章学习：Linux 容器技术

1. 阅读范围与问题主线

本周阅读第 11 章“Linux 容器技术”，印刷页 **309—353**，对应扫描 PDF 的第 **328—372** 页。教材把容器定义为操作系统级虚拟化：一组特殊进程，通过 namespace、cgroup 等技术把资源、文件、设备和配置划分到独立空间。

教材部分	印刷页	本周重点解决的问题
11.1 容器技术概述	309—311	容器与虚拟机差在哪里？为什么能共享内核？
11.2 容器技术的基本原理	311—331	chroot、namespace、cgroup 各自解决什么问题？
11.3 如何创建一个容器	331—334	哪些系统调用参与创建？OCI 包怎样运行？
11.4 Docker	334—338	镜像、Dockerfile 与运行时是什么关系？
11.5 iSula	338—353	容器引擎的架构与基本用法

2. 问题一：容器为什么不叫虚拟机？

疑问与原因。 两者都能提供“隔离的运行环境”，都能打包应用和依赖，初学者容易把容器当成“更轻的虚拟机”。

理解与解决。 根本区别在**是否共享内核**：

维度	虚拟机	容器
隔离层次	硬件级，运行完整客户机操作系统	操作系统级，共享宿主机内核
启动	需要引导客户机内核	直接创建进程，启动快
资源开销	需要独立内核、虚拟设备	接近普通进程
隔离强度	强，边界是虚拟化层	较弱，边界是内核提供的 namespace/cgroup
兼容性	可运行不同内核/架构的客户机	必须与宿主机内核和架构兼容

教材特别指出：容器没有虚拟化技术支撑时，ARM 容器只能运行在 ARM 主机上，x86 容器只能运行在 x86 主机上。这不是实现细节，而是“共享内核”的直接结果。

3. 问题二：`chroot` 能提供隔离吗？

疑问与原因。 `chroot` 能把进程的根目录换到一个子目录，看起来已经“关起来”了；为什么还需要 namespace？

理解与解决。 `chroot` 只改变进程看到的**根目录**，不改变其他资源视图：

- 进程仍能看到宿主机的主机名、进程号、网络接口；
- 具有足够权限的进程可能通过已打开的描述符、`chdir("..")`、挂载等路径逃逸；
- 它不限制 CPU、内存等资源使用。

因此 `chroot` 是容器文件系统视图的早期基础，但不是完整的隔离机制，更不能当成安全边界。教材把它放在 11.2.1，作为理解“文件系统隔离”的起点，而不是容器隔离的全部。

4. 问题三：namespace 分别隔离什么？

疑问与原因。 容器里能看到独立的 PID、网络和主机名，但这些“独立”是由多个 namespace 共同完成的，容易记混。

理解与解决。 教材 11.2.2 按资源类型逐个介绍：

namespace	隔离内容	容器中的直观表现
Mount	挂载点视图	容器有自己的根文件系统
PID	进程号空间	容器内进程从 1 开始编号
IPC	消息队列、信号量、共享内存	容器间不能直接使用同一 IPC 对象
UTS	主机名、域名	容器可以有自己的 hostname
Network	网络设备、协议栈、端口	容器有自己的 `eth0` 和端口空间
User	用户/组 ID 映射	容器内 root 可映射为宿主机普通用户
Cgroup	cgroup 视图	容器内看到自己的资源层级

这些 namespace 通过 `clone`/`unshare`/`setns` 等系统调用创建或加入。一个容器通常同时使用多个 namespace；只创建其中一个，并不能得到完整的容器视图。

5. 问题四：cgroup 与 namespace 的分工是什么?

疑问与原因。 两者都常与容器一起出现，容易以为 namespace 也能限制资源。

理解与解决。 分工清楚：

- **namespace 管“看见什么”**：隔离视图，让进程看不到其他命名空间的对象；
- **cgroup 管“能用多少”**：限制、统计和优先级，例如 CPU 带宽、内存上限、块 I/O 权重、进程数上限。

> namespace：容器的“边界”
> cgroup：容器的“配额”

只有 namespace，容器可能看不到宿主机进程，但仍可能耗尽 CPU 或内存；只有 cgroup，进程仍能看到宿主机的全部视图。两者结合，才构成“既隔离又受限”的运行环境。这个区分也解释了为什么容器逃逸往往与权限、挂载和内核漏洞有关，而不是单靠 namespace 就能保证安全。

教材 11.3 把“创建一个容器”归纳为一组系统调用和文件系统操作，顺序大致如下：

```
clone(CLONE_NEWNS | CLONE_NEWPID | CLONE_NEWNET | …)
  → 子进程进入新的 namespace
      → mount / pivot_root：准备独立的根文件系统
      → 写入 cgroup：设置 CPU、内存等限制
      → execve：执行容器入口程序
```

`unshare` 可以让当前进程加入新的 namespace，`setns` 可以加入已有 namespace；OCI 运行时（如 `runc`）就是按配置完成这些步骤，而不是调用一个名为“创建容器”的内核接口。理解这一点，就不会把容器当成内核里的一个新对象。

6. 问题五：容器里的 1 号进程与宿主机 PID 1 是什么关系?

疑问与原因。 容器里 `ps` 看到 PID 1，宿主机上也有 PID 1；两个 1 号进程同时存在，容易混淆。

理解与解决。 PID namespace 让 PID 编号按命名空间独立分配。容器内的第一个进程在该 namespace 中获得 PID 1，但它在宿主机上有另一个全局 PID。两者是同一个任务在不同 namespace 中的两种编号，不是两个 PID 1 争夺同一个身份。

PID 1 在容器中有特殊语义：它负责回收孤儿进程、接收信号的方式也受限制。若容器主进程退出，PID namespace 中的其他进程会被终止。这与宿主机 PID 1（通常是 init/systemd）承担的系统级职责不同，不能直接类比。

7. 问题六：从 `task_struct` 看，容器到底是什么?

疑问与原因。 教材把容器描述为“一组特殊的进程”；但容器看起来像一个独立的系统，为什么说它只是进程?

理解与解决。 因为容器没有自己的内核，也没有独立的 `task_struct` 类型。每个容器进程仍然是普通的 `task_struct`，只是：

- `nsproxy` 指向一组属于该容器的 namespace；
- `cgroups` 把它挂在特定的资源控制组；
- 根目录、挂载点、网络设备等由 namespace 和挂载配置决定；
- 调度、内存管理、文件系统仍由宿主机内核统一完成。

```
容器 ≈ 一组 task_struct
       + 独立的 namespace 视图
       + cgroup 资源限制
       + 独立的根文件系统
```

这与第 5 章的 `task_struct`、第 6 章的 `fork/clone`、第 8 章的调度、第 9 章的内核共享映射全部连得上：容器不是新物种，而是用已有内核机制组合出来的隔离视图。理解这一点，就不会把容器神秘化，也不会误以为它提供与虚拟机同等级别的隔离。

8. 问题七：Docker、OCI 与容器运行时是什么关系?

疑问与原因。 Docker 既像命令行工具，又像镜像格式，还涉及 `runc`、`containerd`，层次容易混。

理解与解决。 可以按“镜像—运行时—引擎”分层：

层	作用	例子
镜像/包	描述根文件系统与配置	OCI Image、Dockerfile 构建产物
运行时	创建 namespace/cgroup、挂载根文件系统、执行入口	runc、crun
引擎	镜像管理、API、网络、存储、编排	Docker、containerd、iSulad

Docker 早期把引擎和运行时合在一起，后来按 OCI 标准拆分：镜像格式和运行时规范由 OCI 定义，`runc` 负责真正创建容器进程。教材 11.4 讲 Docker 的镜像与 Dockerfile，11.5 讲 iSula，都是在不同层次上实现同一组内核机制。

四、课程收尾：从内核机制到云原生

第 10、11 章把课程从“单个进程怎样运行”推到“多个系统怎样共享硬件”。虚拟机和容器是两条不同路线：

- 虚拟机通过硬件辅助虚拟化，给客户机一个完整的“假硬件”，隔离强、开销大；
- 容器通过 namespace、cgroup 和根文件系统，把一组进程关进独立视图，启动快、开销小，但共享内核。

两条路线都建立在前面几周的机制上：CPU 虚拟化依赖特权级与中断，内存虚拟化依赖页表与地址翻译，容器依赖 `task_struct`、`clone`、调度和文件系统。学完这些再看云原生，容器不再是“另一个世界”，而是内核资源管理能力的一种组合方式。

五、本周问题、原因与解决

问题	原因分析	处理与验证
容器能否替代虚拟机?	混淆隔离层次与隔离强度	容器共享内核，隔离弱于虚拟机，按场景选择
`chroot` 是否足够安全?	只改变根目录视图	还需 namespace、cgroup、权限与挂载控制
namespace 是否限制 CPU?	把“视图隔离”当成“资源限制”	资源限制由 cgroup 完成
容器内 PID 1 与宿主 PID 1 是否同一进程?	忽略 PID namespace 的编号空间	同一任务在不同 namespace 有不同编号
KVM 是否等于 QEMU?	把内核接口与用户态设备模型混为一谈	KVM 提供虚拟化接口，QEMU 提供设备与服务
VM exit 是否就是系统调用?	类比过度	前者切换根/非根模式，后者切换特权级

六、AI 辅助学习中的核验记录

向 AI 梳理的问题	需要核验的点	本次形成的结论
容器是否有自己的内核?	启动过程与 `uname` 结果	没有，共享宿主机内核
namespace 与 cgroup 能否互相替代?	隔离视图与资源配额	不能，职责不同
KVM 的 vCPU 是什么?	线程、`KVM_RUN` 与 VM exit	用户态线程循环进入客户机
内存虚拟化为何需要二级页表?	客户机页表与宿主页表的归属	EPT/NPT 由硬件完成第二级翻译
容器内 root 是否是宿主 root?	user namespace 的 ID 映射	可映射为普通用户，权限随之变化
Docker 是否就是容器运行时?	引擎、运行时、镜像标准的层次	Docker 是引擎，runc 等才是运行时

七、课程总结

十周下来，我从“会写 C 程序”推进到能够解释一个程序从磁盘到 CPU 的完整路径：ELF 文件被 `execve` 装载，`start_thread` 准备好用户现场；任务由 `fork`/`clone` 创建，`copy_thread` 构造首次执行的内核栈；系统调用通过 `system_call` 保存现场、分发服务、恢复返回；调度器在合适的时机选择任务，`switch_to` 换栈并交接 CPU；所有这些又建立在启动、中断、地址空间和内核共享映射之上。

如果只用一句话概括这门课给我的改变：我不再满足于“程序运行了”，而会继续追问“哪个任务、哪份现场、哪条路径让它运行”。这套追问方法，比记住任何一个函数名都更有用。

参考资料

1. 孟宁、娄嘉鹏：《庖丁解牛 Linux 操作系统分析》，第 10 章“KVM 及虚拟机技术”（印刷页 291—308）、第 11 章“Linux 容器技术”（印刷页 309—353）。
2. 《Linux 内核分析》MOOC，第九、十周学习内容与实验九。
3. 作者提供的课程与教材配套资料。
4. Linux 3.18.6 源码：启动初始化、进程创建、ELF 装载、调度核心。
5. Linux 内核文档：KVM、Linux 内核文档：cgroup v2、namespaces(7)、cgroups(7)。
6. OCI 运行时规范、QEMU 文档。